

Week 10: Neural Networks: From the Perceptron to Deep Learning

Artificial Intelligence Applications in Engineering (MUH-920 Mühendislikte Yapay Zeka Uygulamaları). Lecture notes

Prof. Dr. Utku Kose

Full Professor, Department of Computer Engineering, Süleyman Demirel University, Isparta, Türkiye
Founding Director, AI Application and Research Center (YAZEM), Süleyman Demirel University
Head of the Computer Science Division, Department of Computer Engineering, Süleyman Demirel University

Additional affiliations: University of North Dakota (USA), Universidad Panamericana (Mexico City, Mexico), Vel Tech University (Chennai, India)

IEEE Senior Member, ACM Professional Member

ORCID 0000-0002-9652-6415 | utkukose.com | github.com/utkukose

utkukose@sdu.edu.tr | utku.kose@und.edu | ukose@up.edu.mx | utkukose@gmail.com

Week overview, study path and tasks: <https://github.com/utkukose/ai-applications-in-engineering-NB-lecture/blob/main/weeks/week-10/README.md>

Lecture page with animations and knowledge checks: <https://utkukose.github.io/ai-applications-in-engineering-NB-lecture/weeks/week-10/lecture.html>

Interactive lab with self-assessment and reflection: <https://utkukose.github.io/ai-applications-in-engineering-NB-lecture/weeks/week-10/lab.html>

Colab notebook with Python step 10 and the hands-on work:

https://colab.research.google.com/github/utkukose/ai-applications-in-engineering-NB-lecture/blob/main/weeks/week-10/NB10_neural_networks.ipynb

Content licensed under CC BY 4.0. This course is updated in line with current developments in the field.

Last update: October 2026.

Overview

Neural networks connect many simple units whose weights are learned from data. The idea goes back to the logical neuron of McCulloch and Pitts and to Rosenblatt's perceptron [1, 2]; its revival came with backpropagation, which trains networks with hidden layers [3], and its present dominance with deep architectures, large datasets and fast hardware [4, 5]. This week builds a perceptron and a multilayer network from scratch with NumPy, derives backpropagation for a small network, and then uses PyTorch to train a network that predicts the electrical output of a combined cycle power plant [6, 7].

Estimated study time: 10 to 12 hours.

Learning outcomes

By the end of the week, students are expected to explain the perceptron learning rule and its limitation on non-separable data, to compute the forward pass and the gradients of a small multilayer network by hand, to choose activation functions, losses and optimisers, to train a network in PyTorch with a validation set and early stopping, to diagnose underfitting and overfitting from learning curves, and to compare a network fairly with the models of Weeks 7 and 8.

Python in this week

These notes close with Python step 10: Matrices, shapes and tensors. It covers matrix products and shapes with NumPy, activation functions on arrays, and PyTorch tensors with automatic differentiation, applied to the XOR network and a one-weight loss [6, 8]. In the course materials, the step is taught at the

start of the Colab notebook, where each snippet runs in a cell next to its explanation and short exercises give immediate feedback, and later sections of the notebook mark further constructs as Python moves.

From neurons to perceptrons

McCulloch and Pitts described an idealised neuron that sums weighted binary inputs and fires when the sum exceeds a threshold, and showed that networks of such units can compute logical functions [1]. Rosenblatt's perceptron added learning: After each example, the weights move in the direction that corrects a wrong output, $w = w + \eta (y - \hat{y}) x$ [2]. If the two classes can be separated by a straight line, or a hyperplane in more dimensions, the rule finds a separating line in a finite number of steps.

Minsky and Papert analysed what single-layer perceptrons cannot do [9]. The exclusive or, XOR, is the classic example: The points (0, 0) and (1, 1) belong to one class and (0, 1) and (1, 0) to the other, and no straight line separates them. Engineering data are full of such interactions, for example when a fault appears only if temperature is high and speed is low. The remedy is a hidden layer that transforms the inputs into a space where the classes become separable.

Check your understanding. Why can a single perceptron not learn XOR?

- A. The learning rate is always too large
- B. The two classes of XOR cannot be separated by a single straight line
- C. XOR has too many inputs
- D. The perceptron has no bias term

Answer: B. A single threshold unit draws one linear boundary, and XOR needs at least two [9].

Formulas: The threshold neuron and the perceptron rule

A threshold neuron multiplies its inputs $x_1, \dots, x_n$ by weights $w_1, \dots, w_n$ and fires when the weighted sum reaches a threshold [1]. Writing the threshold as a negative bias b , the perceptron computes a score z and a binary output $\hat{y}$:

$$z = w_1 x_1 + w_2 x_2 + \dots + w_n x_n + b = w \cdot x + b$$

$$\hat{y} = 1 \text{ if } z \geq 0, \hat{y} = 0 \text{ if } z < 0$$

The output changes where $z = 0$, so the decision boundary $w \cdot x + b = 0$ is a straight line for two inputs and a plane or hyperplane for more. The weight vector w is perpendicular to the boundary, and an input lies at the distance $|z| / \|w\|$ from it. Part A of the [interactive lab](#) shows this boundary for a single neuron with adjustable weights, bias and activation.

Rosenblatt's learning rule visits the training examples one at a time [2]. For an example with the target y , the output $\hat{y}$ and the error $e = y - \hat{y}$, it changes every weight and the bias with a learning rate $\eta > 0$:

$$w_j \leftarrow w_j + \eta e x_j, b \leftarrow b + \eta e$$

A correct output gives $e = 0$ and no change, a missed positive example gives $e = 1$, and a false alarm gives $e = -1$. Inserting the new weights into the score of the same input shows why the rule corrects mistakes:

$$z_{\text{new}} = (w + \eta e x) \cdot x + b + \eta e = z + \eta e (\|x\|^2 + 1)$$

The score of the input rises after a missed positive and falls after a false alarm, although one correction may not be enough to change the output. If a hyperplane separates the two classes, the perceptron convergence theorem guarantees that the corrections stop after a finite number of mistakes. If no hyperplane separates them, every pass through the data contains a mistake, and the weights never settle [9].

Worked example: A perceptron learns the AND function

A two-hand control releases a press only when both of its buttons are pressed, which is the AND function of two binary inputs. A perceptron starts with $w = (0.5, 0.5)$ and $b = 0.1$. Its scores for the inputs (0, 0), (0, 1), (1, 0) and (1, 1) are 0.1, 0.6, 0.6 and 1.1, so it fires for every input. It learns with $\eta = 0.2$ and visits the examples in this order, with the targets 0, 0, 0 and 1.

Step 1. Input (0, 0), target 0. The score is $z = 0.5 \times 0 + 0.5 \times 0 + 0.1 = 0.1 \geq 0$, so $\hat{y} = 1$ and $e = 0 - 1 = -1$, a false alarm. The inputs are zero, so only the bias changes:

$$w_1 = 0.5 + 0.2 \times (-1) \times 0 = 0.5, w_2 = 0.5 + 0.2 \times (-1) \times 0 = 0.5$$
$$b = 0.1 + 0.2 \times (-1) = -0.1$$

Step 2. Input (0, 1), target 0. The score is $z = 0.5 \times 0 + 0.5 \times 1 - 0.1 = 0.4 \geq 0$, another false alarm with $e = -1$:

$$w_1 = 0.5 + 0.2 \times (-1) \times 0 = 0.5, w_2 = 0.5 + 0.2 \times (-1) \times 1 = 0.3$$
$$b = -0.1 + 0.2 \times (-1) = -0.3$$

By the formula above, the score of this input falls by $\eta(\|x\|^2 + 1) = 0.2 \times (1 + 1) = 0.4$, from 0.4 to exactly 0, which still fires. One correction is not enough here.

Step 3. Input (1, 0), target 0. The score is $z = 0.5 \times 1 + 0.3 \times 0 - 0.3 = 0.2 \geq 0$, a third false alarm:

$$w_1 = 0.5 + 0.2 \times (-1) \times 1 = 0.3, w_2 = 0.3 + 0.2 \times (-1) \times 0 = 0.3$$
$$b = -0.3 + 0.2 \times (-1) = -0.5$$

Step 4. Input (1, 1), target 1. The score is $z = 0.3 \times 1 + 0.3 \times 1 - 0.5 = 0.1 \geq 0$, so $\hat{y} = 1 = y$, and nothing changes.

Step 5. The second pass. With $w = (0.3, 0.3)$ and $b = -0.5$, every example is classified correctly, so training stops after a full pass without a mistake:

Input	Score z	Output	Target
(0, 0)	$0.3 \times 0 + 0.3 \times 0 - 0.5 = -0.5$	0	0
(0, 1)	$0.3 \times 0 + 0.3 \times 1 - 0.5 = -0.2$	0	0
(1, 0)	$0.3 \times 1 + 0.3 \times 0 - 0.5 = -0.2$	0	0
(1, 1)	$0.3 \times 1 + 0.3 \times 1 - 0.5 = 0.1$	1	1

The boundary $0.3x_1 + 0.3x_2 - 0.5 = 0$, or $x_1 + x_2 = 5/3 \approx 1.6667$, leaves only (1, 1) on the firing side. Part B of the [interactive lab](#) replays these steps and tries other functions.

Worked example: Why the exclusive or needs a hidden layer

Step 1. No single perceptron computes it. Suppose that weights w_1, w_2 and a bias b computed the exclusive or, XOR. The outputs 0 for (0, 0) and (1, 1) and the outputs 1 for (0, 1) and (1, 0) would require, in this order,

$$b < 0, w_1 + w_2 + b < 0, w_2 + b \geq 0, w_1 + b \geq 0$$

Adding the last two inequalities gives $w_1 + w_2 + 2b \geq 0$, while adding the first two gives $w_1 + w_2 + 2b < 0$. Both cannot hold, so no such weights exist [9].

Step 2. Two hidden units. The network of the notebook has a hidden unit h_1 with both weights equal to 1 and the bias -0.5 , which computes OR, and a hidden unit h_2 with both weights equal to 1 and the bias -1.5 , which computes AND. With step activations, the hidden layer gives

x_1	x_2	$z_1 = x_1 + x_2 - 0.5$	h_1	$z_2 = x_1 + x_2 - 1.5$	h_2
0	0	$0+0-0.5=-0.5$	0	$0+0-1.5=-1.5$	0
0	1	$0+1-0.5=0.5$	1	$0+1-1.5=-0.5$	0
1	0	$1+0-0.5=0.5$	1	$1+0-1.5=-0.5$	0
1	1	$1+1-0.5=1.5$	1	$1+1-1.5=0.5$	1

Step 3. The output unit. The output unit has the weights 1 and -1 and the bias -0.5 :

Input	h_1	h_2	$z = h_1 - h_2 - 0.5$	$\hat{y}$	XOR
(0, 0)	0	0	$0-0-0.5=-0.5$	0	0
(0, 1)	1	0	$1-0-0.5=0.5$	1	1
(1, 0)	1	0	$1-0-0.5=0.5$	1	1
(1, 1)	1	1	$1-1-0.5=-0.5$	0	0

Step 4. What the hidden layer does. In the plane of (h_1, h_2) , the four inputs become three points: (0, 0) for the input (0, 0), (1, 0) for both (0, 1) and (1, 0), and (1, 1) for the input (1, 1). The line $h_1 - h_2 - 0.5 = 0$ separates the point (1, 0) from the other two, so the output unit, an ordinary perceptron, finishes the job. The hidden layer has transformed the inputs into a space in which the classes are linearly separable. Part B of the lab shows the perceptron rule failing on XOR, and Part C trains a network that finds such a transformation by itself.

Multilayer networks and backpropagation

A multilayer perceptron stacks layers of units. Each unit computes a weighted sum of the outputs of the previous layer and applies a nonlinear activation function: the logistic sigmoid, the hyperbolic tangent, or the rectified linear unit, ReLU, which returns $\max(0, z)$ and eases the training of deep networks [10]. Without nonlinear activations, any stack of layers collapses into one linear map. With them, a network with one sufficiently wide hidden layer can approximate any continuous function on a bounded domain to any accuracy [11, 12], although the theorem says nothing about how many units are needed or whether training finds them.

Training minimises a loss, such as the mean squared error for regression or the cross-entropy for classification, by gradient descent. Backpropagation computes the gradient of the loss with respect to every weight efficiently by applying the chain rule layer by layer from the output back to the input [3]. Each weight then moves a small step against its gradient. Modern frameworks such as PyTorch record the operations of the forward pass and compute these gradients automatically [6].

Animation: A small network learns a curved boundary. A network with one hidden layer is trained by gradient descent on points from two classes. Change the number of hidden units and the learning rate, and watch the boundary bend while the loss falls. Run it on the [lecture page](#).

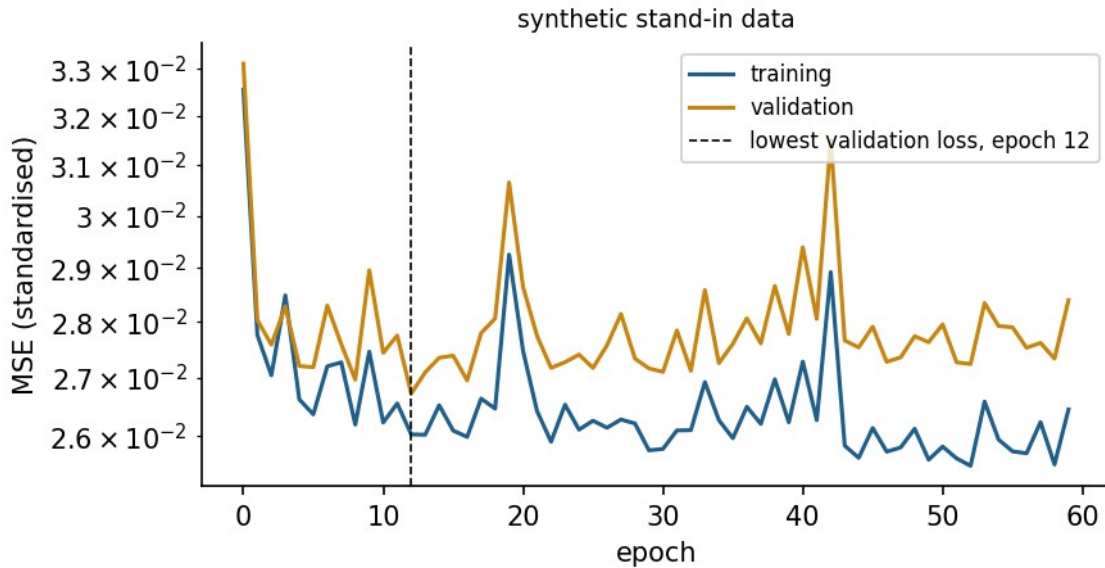


Figure 10.1. Training and validation loss of a PyTorch network for the power plant data. Early stopping keeps the weights from the epoch with the lowest validation loss.

Check your understanding. What does backpropagation compute?

- A. The optimal number of hidden units
- B. The gradient of the loss with respect to all weights, using the chain rule
- C. The training data for the next epoch
- D. The activation function

Answer: B. Backpropagation is an efficient application of the chain rule that yields all partial derivatives in one backward pass [3].

Formulas: Layers, activation functions and backpropagation

A layer l receives the outputs $a_k^{(l-1)}$ of the previous layer, where $a^{(0)} = x$ is the input. Each unit j of the layer forms a weighted sum with its weights $w_{jk}^{(l)}$ and its bias $b_j^{(l)}$ and applies an activation function g :

$$z_j^{(l)} = \sum_k w_{jk}^{(l)} a_k^{(l-1)} + b_j^{(l)}, a_j^{(l)} = g(z_j^{(l)})$$

The weight $w_{jk}^{(l)}$ connects unit k of layer $l-1$ to unit j of layer l . The common activation functions are the logistic sigmoid, the hyperbolic tangent and the rectified linear unit [5, 10]:

$$\sigma(z) = \frac{1}{1+e^{-z}}, \tanh z = \frac{e^z - e^{-z}}{e^z + e^{-z}}, \text{ReLU}(z) = \max(0, z)$$

Backpropagation needs their derivatives. For the sigmoid, the quotient rule and $e^{-z}/(1+e^{-z}) = 1 - \sigma(z)$ give

$$\sigma'(z) = \frac{e^{-z}}{(1+e^{-z})^2} = \frac{1}{1+e^{-z}} \times \frac{e^{-z}}{1+e^{-z}} = \sigma(z)(1 - \sigma(z))$$

and for the other two

$$\frac{d}{dz} \tanh z = 1 - \tanh^2 z, \text{ReLU}'(z) = 1 \text{ for } z > 0 \text{ and } 0 \text{ for } z < 0$$

with the value 0 used in practice at $z=0$. The sigmoid has its largest slope, 0.25, at $z=0$, and the slopes of the sigmoid and of tanh approach zero for large $|z|$, where the unit saturates. ReLU passes the gradient unchanged wherever the unit is active.

Without an activation function, two layers with the weight matrices W_1, W_2 and the bias vectors b_1, b_2 collapse into a single linear map:

$$W_2(W_1x + b_1) + b_2 = (W_2W_1)x + (W_2b_1 + b_2)$$

Stacking more linear layers adds nothing. With a nonlinear activation, one hidden layer of N units computes

$$f(x) = \sum_{j=1}^N c_j g(w_j \cdot x + b_j)$$

and Cybenko and Hornik and colleagues proved that such sums with sigmoid-shaped activations approximate any continuous function on a bounded domain as closely as desired when N is large enough [11, 12].

Training minimises a loss averaged over the n training examples, usually the mean squared error for regression and, for an output probability $\hat{p}$, the cross-entropy of Week 8:

$$L = \frac{1}{n} \sum_{i=1}^n (\hat{y}_i - y_i)^2$$

$$L = -\frac{1}{n} \sum_{i=1}^n [y_i \ln \hat{p}_i + (1 - y_i) \ln (1 - \hat{p}_i)]$$

Backpropagation computes the derivative of the loss with respect to every weight with the chain rule [3]. The central quantity is the error signal $\delta_j = \partial L / \partial z_j$ of each unit, the sensitivity of the loss to the weighted sum of the unit. For an output unit,

$$\delta_j = \frac{\partial L}{\partial a_j} g'(z_j)$$

and a hidden unit j of layer l , whose output feeds the units i of layer $l+1$, collects the error signals of these units through the connecting weights:

$$\delta_j^{(l)} = g'(z_j^{(l)}) \sum_i w_{ij}^{(l+1)} \delta_i^{(l+1)}$$

The gradient of a weight is the error signal of the unit that the weight enters times the activation that it carries, and the gradient of a bias is the error signal itself:

$$\frac{\partial L}{\partial w_{jk}^{(l)}} = \delta_j^{(l)} a_k^{(l-1)}, \quad \frac{\partial L}{\partial b_j^{(l)}} = \delta_j^{(l)}$$

The error signals are computed from the output backwards, which gives the method its name, and then every parameter takes a gradient descent step $w \leftarrow w - \eta \partial L / \partial w$. For a sigmoid output with the cross-entropy of one example, the output error signal becomes very simple:

$$\delta = \left(-\frac{y}{\hat{p}} + \frac{1-y}{1-\hat{p}} \right) \hat{p}(1-\hat{p})$$

$$\delta = -y(1-\hat{p}) + (1-y)\hat{p} = \hat{p} - y$$

This is the error signal at the output of the notebook's network. Frameworks such as PyTorch record the operations of the forward pass and apply the chain rule automatically [6].

Worked example: One backpropagation step by hand

A network with two inputs, two sigmoid hidden units and one sigmoid output receives the example $x = (0.5, 1)$ with the target $y = 1$. Hidden unit j has the weights w_{j1}, w_{j2} and the bias b_j , and the output unit has the weights v_1, v_2 and the bias b_o :

Unit	Weights	Bias
Hidden unit 1	$w_{11}=0.4, w_{12}=-0.2$	$b_1=0.1$
Hidden unit 2	$w_{21}=0.3, w_{22}=0.6$	$b_2=-0.1$
Output unit	$v_1=0.7, v_2=-0.5$	$b_o=0.2$

The loss of the example is $L = \frac{1}{2}(\hat{y} - y)^2$, where the factor $1/2$ cancels the 2 of the derivative, and the learning rate is $\eta = 0.5$. For this network, the formulas of backpropagation read

$$\delta_o = (\hat{y} - y)\hat{y}(1 - \hat{y}), \delta_j = v_j \delta_o h_j(1 - h_j)$$

$$\frac{\partial L}{\partial v_j} = \delta_o h_j, \frac{\partial L}{\partial w_{jk}} = \delta_j x_k$$

All values are rounded to six decimals, and every later step uses the rounded values.

Step 1. Forward pass through the hidden layer.

$$z_1 = 0.4 \times 0.5 + (-0.2) \times 1 + 0.1 = 0.2 - 0.2 + 0.1 = 0.1$$

$$h_1 = \sigma(0.1) = \frac{1}{1 + e^{-0.1}} = \frac{1}{1 + 0.904837} = 0.524979$$

$$z_2 = 0.3 \times 0.5 + 0.6 \times 1 + (-0.1) = 0.15 + 0.6 - 0.1 = 0.65$$

$$h_2 = \sigma(0.65) = \frac{1}{1 + e^{-0.65}} = \frac{1}{1 + 0.522046} = 0.65701$$

Step 2. Output and loss.

$$z_o = 0.7 \times 0.524979 + (-0.5) \times 0.65701 + 0.2$$

$$z_o = 0.367485 - 0.328505 + 0.2 = 0.23898$$

$$\hat{y} = \sigma(0.23898) = \frac{1}{1 + e^{-0.23898}} = \frac{1}{1 + 0.787431} = 0.559462$$

$$L = \frac{1}{2}(0.559462 - 1)^2 = \frac{1}{2} \times (-0.440538)^2$$

$$L = \frac{1}{2} \times 0.194074 = 0.097037$$

Step 3. Error signal of the output. The derivative of the loss is $\partial L / \partial \hat{y} = \hat{y} - y = -0.440538$, and the slope of the sigmoid is $\hat{y}(1 - \hat{y}) = 0.559462 \times 0.440538 = 0.246464$, so

$$\delta_o = -0.440538 \times 0.246464 = -0.108577$$

Step 4. Gradients of the output unit.

$$\frac{\partial L}{\partial v_1} = \delta_o h_1 = -0.108577 \times 0.524979 = -0.057001$$

$$\frac{\partial L}{\partial v_2} = \delta_o h_2 = -0.108577 \times 0.65701 = -0.071336$$

and $\partial L / \partial b_o = \delta_o = -0.108577$.

Step 5. Error signals of the hidden units. The slopes of the hidden units are $h_1(1 - h_1) = 0.524979 \times 0.475021 = 0.249376$ and $h_2(1 - h_2) = 0.65701 \times 0.34299 = 0.225348$. The output error travels back through the output weights as they were before the update:

$$\delta_1 = 0.7 \times (-0.108577) \times 0.249376 = -0.018954$$

$$\delta_2 = (-0.5) \times (-0.108577) \times 0.225348 = 0.012234$$

The second hidden unit receives a positive error signal because its output weight is negative: Increasing its weighted sum would lower $\hat{y}$ and raise the loss.

Step 6. Gradients of the hidden units. With $x_1 = 0.5$ and $x_2 = 1$:

Parameter	Gradient
w_{11}	$\delta_1 x_1 = -0.018954 \times 0.5 = -0.009477$
w_{12}	$\delta_1 x_2 = -0.018954 \times 1 = -0.018954$
b_1	$\delta_1 = -0.018954$
w_{21}	$\delta_2 x_1 = 0.012234 \times 0.5 = 0.006117$
w_{22}	$\delta_2 x_2 = 0.012234 \times 1 = 0.012234$
b_2	$\delta_2 = 0.012234$

Step 7. The update. Every parameter moves against its gradient with $\eta = 0.5$:

Parameter	Old value	Gradient	New value
w_{11}	0.4	-0.009477	$0.4 - 0.5 \times (-0.009477) = 0.4047385$
w_{12}	-0.2	-0.018954	$-0.2 - 0.5 \times (-0.018954) = -0.190523$
b_1	0.1	-0.018954	$0.1 - 0.5 \times (-0.018954) = 0.109477$
w_{21}	0.3	0.006117	$0.3 - 0.5 \times 0.006117 = 0.2969415$
w_{22}	0.6	0.012234	$0.6 - 0.5 \times 0.012234 = 0.593883$
b_2	-0.1	0.012234	$-0.1 - 0.5 \times 0.012234 = -0.106117$
v_1	0.7	-0.057001	$0.7 - 0.5 \times (-0.057001) = 0.7285005$
v_2	-0.5	-0.071336	$-0.5 - 0.5 \times (-0.071336) = -0.464332$
b_o	0.2	-0.108577	$0.2 - 0.5 \times (-0.108577) = 0.2542885$

Step 8. The effect. A forward pass with the new parameters, with the products rounded to six decimals, gives

$$z_1 = 0.4047385 \times 0.5 + (-0.190523) \times 1 + 0.109477$$

$$z_1 = 0.202369 - 0.190523 + 0.109477 = 0.121323$$

$$h_1 = \frac{1}{1 + e^{-0.121323}} = \frac{1}{1 + 0.885748} = 0.530294$$

$$z_2 = 0.2969415 \times 0.5 + 0.593883 \times 1 + (-0.106117)$$

$$z_2 = 0.148471 + 0.593883 - 0.106117 = 0.636237$$

$$h_2 = \frac{1}{1 + e^{-0.636237}} = \frac{1}{1 + 0.52928} = 0.653902$$

$$z_o = 0.7285005 \times 0.530294 + (-0.464332) \times 0.653902 + 0.2542885$$

$$z_o = 0.386319 - 0.303628 + 0.2542885 = 0.3369795$$

$$\hat{y} = \frac{1}{1 + e^{-0.3369795}} = \frac{1}{1 + 0.713923} = 0.583457$$

$$L = \frac{1}{2} \times (-0.416543)^2 = 0.086754$$

The loss has fallen from 0.097037 to 0.086754, and the output has moved from 0.559462 towards the target 1, to 0.583457.

Step 9. A numerical check. A gradient can be checked with a central difference, whose error shrinks with the square of the step h :

$$\frac{\partial L}{\partial v_1} \approx \frac{L(v_1 + h) - L(v_1 - h)}{2h}$$

Recomputing the loss without rounding for $v_1 = 0.701$ and $v_1 = 0.699$, with $h = 0.001$, gives $L = 0.0969797332$ and $L = 0.0970937343$, so that

$$\frac{0.0969797332 - 0.0970937343}{0.002} = \frac{-0.0001140011}{0.002} = -0.0570005$$

which agrees with $\delta_o h_1 = -0.057001$ from Step 4. The notebook checks a gradient of its XOR network in the same way, and Part D of the [interactive lab](#) repeats this example number by number for any weights.

Optimisation in practice

Plain gradient descent uses the whole training set for every step. Stochastic gradient descent uses small random mini-batches, which is faster and adds noise that can help to escape poor regions. Momentum accumulates a moving average of gradients and damps oscillation across narrow valleys. Adam adapts the step size of every weight from running estimates of the first and second moments of its gradient and is a robust default [13]. The learning rate remains the most important setting: Too small and training crawls, too large and the loss oscillates or diverges.

Initialisation matters because signals must neither vanish nor explode as they pass through many layers. Glorot and Bengio proposed scaling the initial weights with the number of inputs and outputs of a layer [14]. Batch normalisation standardises the inputs of a layer over each mini-batch and often allows larger learning rates [15]. Inputs of different units, such as temperatures in degrees and pressures in millibars, should be standardised before training, as for k-means and support vector machines.

Formulas: Gradient descent, momentum, Adam, initialisation and normalisation

Training minimises the average loss $L(\theta) = \frac{1}{n} \sum_i \ell_i(\theta)$ of the n training examples, where θ collects all weights and biases and ℓ_i is the loss of example i . Gradient descent takes steps against the gradient ∇L , the vector of all partial derivatives. Stochastic gradient descent estimates the gradient from a random mini-batch B of $|B|$ examples [5]:

$$g = \frac{1}{|B|} \sum_{i \in B} \nabla \ell_i(\theta), \theta \leftarrow \theta - \eta g$$

One pass through all mini-batches is an epoch, so an epoch takes $n/|B|$ steps. Momentum keeps a velocity v , a decaying sum of past steps with a factor α , typically 0.9 [3, 5]:

$$v \leftarrow \alpha v - \eta g, \theta \leftarrow \theta + v$$

Steps that point the same way accumulate, and steps that alternate across a narrow valley cancel. Adam keeps running means of the gradient and of its square for every parameter, corrects their bias towards zero in the first steps, and divides the first by the square root of the second [13]:

$$m \leftarrow \beta_1 m + (1 - \beta_1) g, s \leftarrow \beta_2 s + (1 - \beta_2) g^2$$
$$\hat{m} = \frac{m}{1 - \beta_1^t}, \hat{s} = \frac{s}{1 - \beta_2^t}, \theta \leftarrow \theta - \eta \frac{\hat{m}}{\sqrt{\hat{s} + \epsilon}}$$

Here t counts the steps, m and s start at zero, and the defaults are $\beta_1 = 0.9$, $\beta_2 = 0.999$ and $\epsilon = 10^{-8}$. All operations act on every parameter separately.

The learning rate decides whether gradient descent converges at all. For the loss $L(w) = a(w - w^*)^2$ with $a > 0$, the gradient is $2a(w - w^*)$, and one step changes the error $e = w - w^*$ to

$$e \leftarrow e - 2a\eta e = (1 - 2a\eta)e$$

The error shrinks steadily for $0 < \eta \leq 1/(2a)$, shrinks with an alternating sign for $1/(2a) < \eta < 1/a$, and does not shrink for $\eta \geq 1/a$. In many dimensions, the steepest direction limits the learning rate, while the flattest direction sets the speed, which is the difficulty of the narrow valley in Part E of the lab.

Initialisation sets the scale of the signals. For a weighted sum $z = \sum_{k=1}^{n_{\text{in}}} w_k a_k$ of independent inputs with mean zero and independent weights with mean zero, the variances multiply:

$$\text{Var}(z) = n_{\text{in}} \text{Var}(w) \text{Var}(a)$$

Keeping the variance of the signals from layer to layer requires $\text{Var}(w) = 1/n_{\text{in}}$, and keeping the variance of the error signals on the way back requires $\text{Var}(w) = 1/n_{\text{out}}$, where n_{out} is the number of units of the layer. Glorot and Bengio proposed the compromise [14]

$$\text{Var}(w) = \frac{2}{n_{\text{in}} + n_{\text{out}}}, w \sim U(-r, r) \text{ with } r = \sqrt{\frac{6}{n_{\text{in}} + n_{\text{out}}}}$$

since a uniform distribution on the interval from $-r$ to r has the variance $r^2/3$.

Batch normalisation standardises the weighted sums $x_1, \dots, x_m$ of a unit over each mini-batch of m examples and then rescales them with two learned parameters, γ and β [15]:

$$\mu_B = \frac{1}{m} \sum_{i=1}^m x_i, \sigma_B^2 = \frac{1}{m} \sum_{i=1}^m (x_i - \mu_B)^2$$

$$\hat{x}_i = \frac{x_i - \mu_B}{\sqrt{\sigma_B^2 + \epsilon}}, y_i = \gamma \hat{x}_i + \beta$$

The small constant ϵ prevents a division by zero. At test time, running averages of μ_B and σ_B^2 collected during training take the place of the statistics of the batch.

Worked example: Three optimisers on one loss

The loss $L(w) = (w - 3)^2$ has its minimum at $w = 3$ and the gradient $g = 2(w - 3)$. All three optimisers start at $w_0 = 0$ with $\eta = 0.1$. Momentum uses $\alpha = 0.9$, and Adam uses its defaults.

Step 1. Gradient descent.

$$g_1 = 2 \times (0 - 3) = -6, w_1 = 0 - 0.1 \times (-6) = 0.6$$

$$g_2 = 2 \times (0.6 - 3) = -4.8, w_2 = 0.6 - 0.1 \times (-4.8) = 1.08$$

Step 2. Momentum. With $v_0 = 0$, the first step equals that of gradient descent, so the second gradient is again $g_2 = -4.8$:

$$v_1 = 0.9 \times 0 - 0.1 \times (-6) = 0.6, w_1 = 0 + 0.6 = 0.6$$

$$v_2 = 0.9 \times 0.6 - 0.1 \times (-4.8) = 0.54 + 0.48 = 1.02$$

$$w_2 = 0.6 + 1.02 = 1.62$$

Step 3. Adam, first step. With $g_1 = -6$ and $g_1^2 = 36$:

$$m_1 = 0.9 \times 0 + 0.1 \times (-6) = -0.6, s_1 = 0.999 \times 0 + 0.001 \times 36 = 0.036$$

$$\hat{m}_1 = \frac{-0.6}{1 - 0.9} = -6, \hat{s}_1 = \frac{0.036}{1 - 0.999} = 36$$

$$w_1 = 0 - 0.1 \times \frac{-6}{\sqrt{36 + 10^{-8}}} = 0.1$$

The bias correction undoes the start at zero, and the first step has the length η whatever the size of the gradient, because $\hat{m}_1 / \sqrt{\hat{s}_1} = g_1 / |g_1|$.

Step 4. Adam, second step. Now $g_2 = 2 \times (0.1 - 3) = -5.8$ and $g_2^2 = 33.64$:

$$m_2 = 0.9 \times (-0.6) + 0.1 \times (-5.8) = -0.54 - 0.58 = -1.12$$

$$s_2 = 0.999 \times 0.036 + 0.001 \times 33.64 = 0.035964 + 0.03364 = 0.069604$$

$$\hat{m}_2 = \frac{-1.12}{1 - 0.9^2} = \frac{-1.12}{0.19} = -5.894737$$

$$\hat{s}_2 = \frac{0.069604}{1 - 0.999^2} = \frac{0.069604}{0.001999} = 34.81941$$

With $\sqrt{34.81941} = 5.900797$, the step is

$$w_2 = 0.1 - 0.1 \times \frac{-5.894737}{5.900797} = 0.1 - 0.1 \times (-0.998973) = 0.199897$$

Step 5. Comparison. After two steps:

Optimiser	w_2	$L(w_2) = (w_2 - 3)^2$
Gradient descent	1.08	$(-1.92)^2 = 3.6864$
Momentum	1.62	$(-1.38)^2 = 1.9044$
Adam	0.199897	$(-2.800103)^2 = 7.840577$

Momentum is fastest here because its steps accumulate. Adam is slowest because its steps have a length of about $\eta = 0.1$, so with Adam the learning rate is roughly the largest change of a weight in one step [13].

Step 6. The learning rate. Here $a = 1$, so the error factor is $1 - 2\eta$. With $\eta = 0.1$, the factor 0.8 turns the error $e_0 = 0 - 3 = -3$ into -2.4 and -1.92 , the values of Step 1. With $\eta = 0.75$, the factor is -0.5 : The errors are 1.5 and -0.75 , so w jumps to 4.5 and back to 2.25 while it converges. With $\eta = 1.1$, the factor is -1.2 : The errors are 3.6 and -4.32 , so w goes to 6.6 and then to -1.32 , and the iteration diverges. Part F of the [interactive lab](#) runs all three optimisers on any loss of this form.

Worked example: Initial weights and a normalised mini-batch

Step 1. The Glorot range. The first layer of a network for the eight inputs of the concrete data of Week 7 has 16 units, so $n_{\text{in}} = 8$ and $n_{\text{out}} = 16$:

$$r = \sqrt{\frac{6}{8+16}} = \sqrt{0.25} = 0.5, \text{Var}(w) = \frac{0.5^2}{3} = \frac{1}{12} \approx 0.083333$$

which equals $2/(8+16) = 2/24 = 1/12$.

Step 2. The signal of one unit. For standardised inputs, with the variance 1, the weighted sum of a unit has

$$\text{Var}(z) = 8 \times \frac{1}{12} \times 1 = \frac{2}{3} \approx 0.666667, \sqrt{2/3} = 0.816497$$

A tanh unit whose weighted sum lies one standard deviation from zero, at $z = 0.816497$, has the output $\tanh 0.816497 = 0.673159$ and the slope $1 - 0.673159^2 = 0.546857$, so it passes gradients well. With weights drawn from $U(-3, 3)$ instead, $\text{Var}(w) = 3^2/3 = 3$ and $\text{Var}(z) = 8 \times 3 = 24$, so the standard deviation is $\sqrt{24} = 4.898979$. At this typical value, $\tanh 4.898979 = 0.999889$ and the slope is $1 - 0.999889^2 = 0.000222$: The unit is saturated and passes almost no gradient back.

Step 3. Batch normalisation. One unit produces the weighted sums 2, 4, 6 and 8 for a mini-batch of four examples. With $\epsilon = 10^{-5}$, the default of PyTorch:

$$\mu_B = \frac{2+4+6+8}{4} = \frac{20}{4} = 5$$

$$\sigma_B^2 = \frac{(-3)^2 + (-1)^2 + 1^2 + 3^2}{4} = \frac{9+1+1+9}{4} = 5$$

$$\sqrt{\sigma_B^2 + \epsilon} = \sqrt{5.00001} = 2.23607$$

$$\hat{x}_1 = \frac{-3}{2.23607} = -1.34164, \hat{x}_2 = \frac{-1}{2.23607} = -0.447213$$

and by symmetry $\hat{x}_3=0.447213$ and $\hat{x}_4=1.34164$. With $\gamma=2$ and $\beta=1$:

$$y_1=2 \times (-1.34164)+1=-1.68328, y_2=2 \times (-0.447213)+1=0.105574$$

$$y_3=2 \times 0.447213+1=1.894426, y_4=2 \times 1.34164+1=3.68328$$

The normalised values have the mean 0 and the variance $5/5.00001=0.999998$, so the outputs have the mean $\beta=1$ and a standard deviation very close to $\gamma=2$. Because γ and β are learned, the network can undo the normalisation where that helps. Part G of the lab follows signals through many layers and normalises any mini-batch.

Generalisation and regularisation

Networks with many weights can memorise their training data. The standard defences are a validation set with early stopping, which keeps the weights from the epoch with the lowest validation loss; weight decay, which penalises large weights like ridge regression; and dropout, which randomly switches off units during training so that the network cannot rely on any single unit [16]. Learning curves of training and validation loss over epochs show whether a network underfits, with both losses high, or overfits, with a growing gap.

On tabular data of moderate size, the tree ensembles of Weeks 7 and 8 are often as accurate as networks and easier to tune, so a network must earn its place by a fair comparison. Networks have long been applied in engineering: Yeh modelled concrete strength with them [17], and metaheuristics such as the ant lion optimiser have been used to train them where gradient methods struggle [18]. Their decisive advantages appear with images, signals, sequences and text, where they learn features that would otherwise have to be designed by hand [4]. Weeks 11 to 13 follow this path.

Check your understanding. Training loss keeps falling, while validation loss has been rising for 30 epochs. What should be done?

- A. Train longer with the same settings
- B. Use the weights from the epoch with the lowest validation loss and consider more regularisation
- C. Remove the validation set
- D. Increase the learning rate

Answer: B. A rising validation loss with falling training loss signals overfitting; early stopping and regularisation address it [16].

Formulas: Parameters, weight decay, dropout and early stopping

A fully connected layer with n_{l-1} inputs and n_l units has $n_{l-1}n_l$ weights and n_l biases. A network with the layer sizes $n_0, n_1, \dots, n_K$, from the inputs to the outputs, therefore has

$$P = \sum_{l=1}^K (n_{l-1} + 1) n_l$$

trainable parameters. When P approaches the number of training examples, the network can fit the training data closely, noise included.

Weight decay adds the squared weights to the loss with a factor λ , as ridge regression does in Week 7 [5]:

$$L_\lambda(w) = L(w) + \frac{\lambda}{2} \sum_j w_j^2, \frac{\partial L_\lambda}{\partial w_j} = \frac{\partial L}{\partial w_j} + \lambda w_j$$

A gradient step therefore shrinks every weight by the factor $1 - \eta \lambda$ before it applies the gradient of the data:

$$w_j \leftarrow w_j - \eta \left(\frac{\partial L}{\partial w_j} + \lambda w_j \right) = (1 - \eta \lambda) w_j - \eta \frac{\partial L}{\partial w_j}$$

Weights that the data do not support decay towards zero.

Dropout switches units off at random during training [16]. Each output h_j of a layer is multiplied by an independent mask value m_j , which is 1 with the keep probability q and 0 otherwise. Deep learning libraries such as PyTorch also divide by q , so that the expected value of every output stays the same:

$$\tilde{h}_j = \frac{m_j h_j}{q}, E[\tilde{h}_j] = q \times \frac{h_j}{q} + (1 - q) \times 0 = h_j$$

At test time, all units are used without a mask. Srivastava and colleagues multiplied the outgoing weights by q at test time instead, which gives the same expected values. In PyTorch, the argument p of the dropout layer is the probability of dropping a unit, $p = 1 - q$.

Early stopping computes the validation loss $L_{\text{val}}(e)$ after every epoch e and keeps the weights of the epoch with the lowest value,

$$e^* = \arg \min_e L_{\text{val}}(e)$$

It stops training when the best value has not improved for k consecutive epochs, the patience. The difference $L_{\text{val}} - L_{\text{train}}$, the generalisation gap, grows when the network starts to memorise its training data.

Worked example: Parameters and weight decay of a small network

Step 1. Counting parameters. A network for the concrete data of Week 7, with the seven ingredients and the age as its eight inputs, two hidden layers of 16 units and one output, has the layer sizes 8, 16, 16 and 1:

$$P = (8+1) \times 16 + (16+1) \times 16 + (16+1) \times 1 = 144 + 272 + 17 = 433$$

Step 2. Data per parameter. The data set has 1030 mixes, which gives $1030/433 = 2.38$ examples per parameter. A training split of 80% keeps $0.8 \times 1030 = 824$ mixes and $824/433 = 1.9$ examples per parameter. Such a network can fit noise unless it is regularised and checked on validation data.

Step 3. One step with weight decay. A weight $w = 0.8$ has the gradient $\partial L / \partial w = 0.1$, the learning rate is $\eta = 0.1$, and $\lambda = 0.01$. The shrink factor is $1 - 0.1 \times 0.01 = 0.999$, so

$$w \leftarrow 0.999 \times 0.8 - 0.1 \times 0.1 = 0.7992 - 0.01 = 0.7892$$

Without weight decay, the step would give $0.8 - 0.01 = 0.79$.

Step 4. Decay in the long run. A weight that receives no gradient from the data keeps the fraction 0.999 of its value in every step. After 1000 steps it keeps

$$0.999^{1000} = e^{1000 \ln 0.999} = e^{1000 \times (-0.0010005)} = e^{-1.0005} = 0.367695$$

of its value, close to $e^{-\eta \lambda t} = e^{-0.001 \times 1000} = e^{-1}$ after t steps. Part H of the [interactive lab](#) counts parameters, applies weight decay and draws dropout masks.

Worked example: Dropout and early stopping

Step 1. One dropout mask. A hidden layer has the outputs $h = (0.8, 0.3, 0.5, 0.9)$, and a unit of the next layer has the weights $(0.5, -0.4, 0.3, 0.2)$. At test time, its weighted sum is

$$z = 0.5 \times 0.8 + (-0.4) \times 0.3 + 0.3 \times 0.5 + 0.2 \times 0.9$$

$$z = 0.4 - 0.12 + 0.15 + 0.18 = 0.61$$

During training with $q=0.5$, the mask $m=(1, 0, 1, 0)$ keeps the first and third units and doubles their outputs, $\tilde{h}=(0.8/0.5, 0, 0.5/0.5, 0)=(1.6, 0, 1, 0)$, so

$$\tilde{z} = 0.5 \times 1.6 + 0.3 \times 1 = 0.8 + 0.3 = 1.1$$

Step 2. The expected value. The complementary mask $(0, 1, 0, 1)$ gives $\tilde{h}=(0, 0.6, 0, 1.8)$ and

$$\tilde{z} = (-0.4) \times 0.6 + 0.2 \times 1.8 = -0.24 + 0.36 = 0.12$$

The two masks are equally likely, and their mean $(1.1 + 0.12)/2 = 0.61$ is the test value. The mean over all 16 equally likely masks is also exactly 0.61, since every unit is kept in half of them. Single masks give very different sums, and this noise keeps the network from relying on any single unit.

Step 3. Early stopping. A network is trained with a patience of three epochs:

Epoch	Training loss	Validation loss	Gap	Best so far	Epochs without improvement
1	0.50	0.52	$0.52 - 0.50 = 0.02$	0.52 (epoch 1)	0
2	0.38	0.41	$0.41 - 0.38 = 0.03$	0.41 (epoch 2)	0
3	0.31	0.36	$0.36 - 0.31 = 0.05$	0.36 (epoch 3)	0
4	0.27	0.34	$0.34 - 0.27 = 0.07$	0.34 (epoch 4)	0
5	0.24	0.35	$0.35 - 0.24 = 0.11$	0.34 (epoch 4)	1
6	0.21	0.37	$0.37 - 0.21 = 0.16$	0.34 (epoch 4)	2
7	0.19	0.36	$0.36 - 0.19 = 0.17$	0.34 (epoch 4)	3

After epoch 7, the validation loss has not improved for three epochs, so training stops and the weights of epoch 4, with the validation loss 0.34, are restored. Epoch 7 is better than epoch 6 but not better than the best value, so the counter keeps rising.

Step 4. Reading the curves. The training loss falls in every epoch, while the gap grows from 0.07 to 0.17 after epoch 4: The network has started to fit the noise of its training data. Two high losses close to each other would instead signal underfitting, which calls for a larger network or longer training rather than more regularisation. Part I of the lab applies the rule to any losses and trains a small network until it overfits.

Python step 10: Matrices, shapes and tensors

Vectors, matrices and the @ operator

A layer of a neural network multiplies its inputs by a weight matrix and adds a bias. When a batch of inputs is stored as a matrix X with one example per row, the whole layer is $X @ W + b$, where $@$ is the matrix product. The shapes must match: A matrix of shape (4, 2) times one of shape (2, 2) gives shape (4, 2). NumPy adds the bias vector to every row automatically, a rule called broadcasting. The network below solves XOR with hand-set weights: One hidden unit acts as OR, the other as AND, and the output fires for OR but not for AND.

```
import numpy as np
```

```
X = np.array([[0, 0], [0, 1], [1, 0], [1, 1]]) # four input examples
W1 = np.array([[1, 1], [1, 1]]) # column 0: OR unit,
# column 1: AND unit
b1 = np.array([-0.5, -1.5])
H = (X @ W1 + b1 > 0).astype(int) # step activation
w2, b2 = np.array([1, -1]), -0.5
```

```

y = (H @ w2 + b2 > 0).astype(int)
print(X.shape, W1.shape, H.shape)
print(H)
print(y)

```

Output

```

(4, 2) (2, 2) (4, 2)
[[0 0]
 [1 0]
 [1 0]
 [1 1]]
[0 1 1 0]

```

Activation functions

Smooth activation functions replace the step so that gradients exist. The sigmoid squashes any number into the range 0 to 1, the hyperbolic tangent into -1 to 1, and the rectified linear unit keeps positive values and sets negative ones to zero. Written with NumPy, each of them works on whole arrays.

```

def sigmoid(z):
    return 1 / (1 + np.exp(-z))

```

```

def relu(z):
    return np.maximum(z, 0)

```

```

z = np.array([-2.0, 0.0, 2.0])
print(sigmoid(z).round(3), np.tanh(z).round(3), relu(z))

```

Output

```

[0.119 0.5    0.881] [-0.964  0.    0.964] [0.  0.  2.]

```

Tensors

PyTorch stores data in tensors, which behave much like NumPy arrays but can also run on graphics processors and record the operations applied to them [6]. Neural network code usually works with 32-bit floats, so conversions from NumPy name the type explicitly. `.shape` works as in NumPy, `.item()` extracts a single number, and `.numpy()` converts back to an array.

```

import torch

Xt = torch.tensor(X, dtype=torch.float32)
W1t = torch.tensor(W1, dtype=torch.float32)
print(Xt.shape, Xt.dtype)
print((Xt @ W1t).sum().item(), (Xt @ W1t).numpy().tolist())

```

Output

```

torch.Size([4, 2]) torch.float32
8.0 [[0.0, 0.0], [1.0, 1.0], [1.0, 1.0], [2.0, 2.0]]

```

Gradients with automatic differentiation

Training needs the derivative of the loss with respect to every weight. When a tensor is created with `requires_grad=True`, PyTorch records every operation on it, and `backward` computes the derivatives with the chain rule. For the loss $L = (w \cdot x - y)^2$ with $w = 0.5$, $x = 2$ and $y = 3$, the derivative is

$2(w x - y) x = -8$. A gradient descent step then moves w against the gradient, and `torch.no_grad()` stops the recording while the weight is updated.

```
w = torch.tensor(0.5, requires_grad=True)
x_in, target = torch.tensor(2.0), torch.tensor(3.0)
loss = (w * x_in - target) ** 2
loss.backward()
print(loss.item(), w.grad.item())
with torch.no_grad():
    w -= 0.1 * w.grad          # one gradient descent step with
                               # learning rate 0.1
print(round(w.item(), 4), round(((w * x_in - target) ** 2).item(), 4))
```

Output

```
4.0  -8.0
1.3  0.16
```

The loss falls from 4.0 to about 0.16 in one step. The training loops of this week repeat exactly this pattern for thousands of weights at once.

Check your understanding. A has shape (32, 10) and B has shape (10, 4). What is the shape of A @ B?

- A. (32, 10)
- B. (10, 4)
- C. (32, 4)
- D. The product is not defined

Answer: C. The inner dimensions must agree, and the result keeps the outer ones: 32 rows and 4 columns.

Discipline challenges

Networks shine when relations are nonlinear and data are plentiful. Choose a row and compare a network with the best model of Weeks 7 and 8 under identical splits.

Department	Challenge
Electrical and Electronics Engineering and Mechanical Engineering	Power plant output from ambient conditions, UCI 294, as in the notebook [7].
Civil Engineering	Concrete strength with a network, following Yeh's study [17], compared with gradient boosting.
Civil Engineering and Physics	Heating and cooling loads of buildings as a two-output network, UCI 242 [19].
Environmental Engineering and Chemical Engineering	Gas turbine NOx emissions, UCI 551, with a network and a physics-motivated feature [20].
Physics and Chemistry	Critical temperature of superconductors, UCI 464 [21].
Mechanical Engineering and Industrial Engineering	Failure classification on AI4I with a network and class weights [22].
Computer Engineering and Mathematics	Implement the backward pass of a two-hidden-layer network in NumPy and verify it with numerical gradients.
Food Engineering and Biology	Dry bean classification with a network compared with the random forest of Week 8 [23].
Statistics	Compare early stopping, weight decay and dropout over ten seeds and report the variability of the test error.

Department	Challenge
Mining Engineering, Geology and Earth Sciences Engineering	Facies classification from well logs with a network and a well-wise split [24].

Weekly task

Train a PyTorch network for a regression or classification problem from your department, using the switch of the notebook or a dataset from DATASETS.md. Report the architecture, preprocessing, optimiser, learning curves, early stopping and a fair comparison with a linear model and a tree ensemble on the same test set. Discuss in about 500 words whether the network is worth its complexity, citing at least three works from this week's references [3, 13, 16].

Research and report assignment (optional)

Neural networks for engineering modelling. Review the use of neural networks as surrogate or data-driven models in one engineering field, such as structural response, process modelling, energy prediction or materials properties. Compare network types, training data sizes, validation practices and the baselines used, and discuss when physical models or simpler learners were competitive [4, 17, 25].

Both assignments are optional. During an active semester, the weekly task can be sent together with the learning log exported from the interactive lab, and the research report on its own, to utkukose@sdu.edu.tr or utkukose@gmail.com. They carry no separate weight, but the instructor may take them into account as a discretionary adjustment of the midterm component of the grade. The assessment section of the syllabus gives the details, and research reports follow the report template of the course repository.

References

- [1] McCulloch, W. S., & Pitts, W. (1943). A logical calculus of the ideas immanent in nervous activity. *The Bulletin of Mathematical Biophysics*, 5(4), 115-133. <https://doi.org/10.1007/BF02478259>
- [2] Rosenblatt, F. (1958). The perceptron: A probabilistic model for information storage and organization in the brain. *Psychological Review*, 65(6), 386-408. <https://doi.org/10.1037/h0042519>
- [3] Rumelhart, D. E., Hinton, G. E., & Williams, R. J. (1986). Learning representations by back-propagating errors. *Nature*, 323(6088), 533-536. <https://doi.org/10.1038/323533a0>
- [4] LeCun, Y., Bengio, Y., & Hinton, G. (2015). Deep learning. *Nature*, 521(7553), 436-444. <https://doi.org/10.1038/nature14539>
- [5] Goodfellow, I., Bengio, Y., & Courville, A. (2016). *Deep Learning*. MIT Press.
- [6] Paszke, A., Gross, S., Massa, F., et al. (2019). PyTorch: An imperative style, high-performance deep learning library. In *Advances in Neural Information Processing Systems 32* (pp. 8024-8035). <https://arxiv.org/abs/1912.01703>
- [7] Tüfekci, P. (2014). Prediction of full load electrical power output of a base load operated combined cycle power plant using machine learning methods. *International Journal of Electrical Power & Energy Systems*, 60, 126-140. <https://doi.org/10.1016/j.ijepes.2014.02.027>
- [8] Harris, C. R., Millman, K. J., van der Walt, S. J., et al. (2020). Array programming with NumPy. *Nature*, 585(7825), 357-362. <https://doi.org/10.1038/s41586-020-2649-2>
- [9] Minsky, M., & Papert, S. (1969). *Perceptrons: An Introduction to Computational Geometry*. MIT Press.
- [10] Nair, V., & Hinton, G. E. (2010). Rectified linear units improve restricted Boltzmann machines. In *Proceedings of the 27th International Conference on Machine Learning* (pp. 807-814).
- [11] Cybenko, G. (1989). Approximation by superpositions of a sigmoidal function. *Mathematics of Control, Signals and Systems*, 2(4), 303-314. <https://doi.org/10.1007/BF02551274>

- [12] Hornik, K., Stinchcombe, M., & White, H. (1989). Multilayer feedforward networks are universal approximators. *Neural Networks*, 2(5), 359-366. [https://doi.org/10.1016/0893-6080\(89\)90020-8](https://doi.org/10.1016/0893-6080(89)90020-8)
- [13] Kingma, D. P., & Ba, J. (2015). Adam: A method for stochastic optimization. In *3rd International Conference on Learning Representations (ICLR 2015)*. <https://arxiv.org/abs/1412.6980>
- [14] Glorot, X., & Bengio, Y. (2010). Understanding the difficulty of training deep feedforward neural networks. In *Proceedings of the Thirteenth International Conference on Artificial Intelligence and Statistics, PMLR 9* (pp. 249-256).
- [15] Ioffe, S., & Szegedy, C. (2015). Batch normalization: Accelerating deep network training by reducing internal covariate shift. In *Proceedings of the 32nd International Conference on Machine Learning, PMLR 37* (pp. 448-456). <https://arxiv.org/abs/1502.03167>
- [16] Srivastava, N., Hinton, G., Krizhevsky, A., Sutskever, I., & Salakhutdinov, R. (2014). Dropout: A simple way to prevent neural networks from overfitting. *Journal of Machine Learning Research*, 15(56), 1929-1958.
- [17] Yeh, I.-C. (1998). Modeling of strength of high-performance concrete using artificial neural networks. *Cement and Concrete Research*, 28(12), 1797-1808. [https://doi.org/10.1016/S0008-8846\(98\)00165-3](https://doi.org/10.1016/S0008-8846(98)00165-3)
- [18] Kose, U. (2018). An ant-lion optimizer-trained artificial neural network system for chaotic electroencephalogram (EEG) prediction. *Applied Sciences*, 8(9), 1613. <https://doi.org/10.3390/app8091613>
- [19] Tsanas, A., & Xifara, A. (2012). Accurate quantitative estimation of energy performance of residential buildings using statistical machine learning tools. *Energy and Buildings*, 49, 560-567. <https://doi.org/10.1016/j.enbuild.2012.03.003>
- [20] Kaya, H., Tüfekci, P., & Uzun, E. (2019). Predicting CO and NOx emissions from gas turbines: Novel data and a benchmark PEMS. *Turkish Journal of Electrical Engineering and Computer Sciences*, 27(6), 4783-4796. <https://doi.org/10.3906/elk-1807-87>
- [21] Hamidieh, K. (2018). A data-driven statistical model for predicting the critical temperature of a superconductor. *Computational Materials Science*, 154, 346-354. <https://doi.org/10.1016/j.commatsci.2018.07.052>
- [22] Matzka, S. (2020). Explainable artificial intelligence for predictive maintenance applications. In *2020 Third International Conference on Artificial Intelligence for Industries (AI4I)* (pp. 69-74). IEEE. <https://doi.org/10.1109/AI4I49448.2020.00023>
- [23] Koklu, M., & Ozkan, I. A. (2020). Multiclass classification of dry beans using computer vision and machine learning techniques. *Computers and Electronics in Agriculture*, 174, 105507. <https://doi.org/10.1016/j.compag.2020.105507>
- [24] Hall, B. (2016). Facies classification using machine learning. *The Leading Edge*, 35(10), 906-909. <https://doi.org/10.1190/tle35100906.1>
- [25] Karniadakis, G. E., Kevrekidis, I. G., Lu, L., Perdikaris, P., Wang, S., & Yang, L. (2021). Physics-informed machine learning. *Nature Reviews Physics*, 3(6), 422-440. <https://doi.org/10.1038/s42254-021-00314-5>